Quizz - Time series

Question 1 :

```
import pandas as pd
import numpy as np

Data={'Close':np.random.normal(size=3,loc=1)}
dates_range=pd.to_datetime(['2026-1-1','2026-2-3','2026-3-7'])
df=pd.DataFrame(data=Data,index=dates_range)
print(df)
```

True or False

1. np.random.normal(size=3, loc=1) generates 3 random numbers from a normal distribution with a mean of 1.
2. dates_range contains 3 dates.
3. pd.to_datetime() converts the date strings into pandas datetime values.
4. df contains 3 rows and 1 column named "Close".
5. Running the code multiple times will always produce exactly the same values in the "Close" column.

Answers

1. **True** — size=3 generates three random values, and loc=1 specifies a mean of 1.
2. **True** — the dates are:
   2026-01-01
   2026-02-03
   2026-03-07
3. **True** — pd.to_datetime() converts the strings into datetime objects suitable for a time-series index.
4. **True** — there are 3 dates (rows) and one column, "Close".
5. **False** — the values are randomly generated, so they will generally be different each time you run the code.

Question 2 :

```
import pandas as pd

aapl_dates = pd.to_datetime(["2026-01-05","2026-01-06","2026-01-07","2026-01-08"])
msft_dates = pd.to_datetime(["2026-01-06","2026-01-07","2026-01-08","2026-01-09"])

difference = aapl_dates.difference(msft_dates)
print(difference)
```

True or False

1. aapl_dates.difference(msft_dates) returns the dates that are present in aapl_dates but not in msft_dates.
2. In the example, "2026-01-05" is present in aapl_dates but not in msft_dates.
3. msft_dates.difference(aapl_dates) returns "2026-01-05".
4. df1.index.difference(df2.index) can be used to identify dates that exist in df1 but are missing from df2.
5. .difference() modifies the original Index permanently.

Quizz - Time series

Answers

6. **True** — .difference() returns the elements that are in the first Index but not in the second.
7. **True** — January 5 appears in aapl_dates but not in msft_dates.
8. **False** — msft_dates.difference(aapl_dates) returns 2026-01-09, because that date exists in msft_dates but not in aapl_dates.
9. **True** — this is a useful way to find dates missing from one financial dataset.
10. **False** — .difference() returns a **new Index**; it doesn't modify the original Index.

Question 3 :

```
import pandas as pd

df = pd.DataFrame({
   "date": pd.date_range("2026-01-01", periods=5),
   "close": [100, 105, 102, 108, 120]
})

df["rendement"] = df["close"].pct_change()*100
print(df)
```

True or False

1. pd.date_range("2026-01-01", periods=5) creates five consecutive daily dates.
2. df["close"].pct_change() calculates the percentage change between consecutive closing prices.
3. Multiplying by 100 converts the returns from decimal form into percentage form.
4. The first value of the "rendement" column will be 0.
5. The return on the second row is 5%.

Answers

1. **True** — the dates are January 1 through January 5, 2026.
2. **True** — pct_change() compares each closing price with the previous closing price.
3. **True** — for example, 0.05 × 100 = 5%.
4. **False** — the first value is NaN because there is no previous closing price to compare with.
5. **True** — the calculation is:

$$\frac{105 - 100}{100} \times 100 = 5\%$$

Question 4 :

```
import pandas as pd

df = pd.DataFrame({
   "date": pd.date_range("2026-01-01", periods=10, freq="D"),
   "close": [100, 102, 101, 105, 107, 106, 110, 112, 111, 115]
})

df = df.set_index("date")
weekly=df.resample(“w”).last()
```

Quizz - Time series
print(df)

True or False

1. pd.date_range(..., periods=10, freq="D") creates 10 consecutive daily dates.
2. df.set_index("date") makes the date column the index of the DataFrame.
3. df.resample("W").last() converts the daily data into weekly data and keeps the last observation of each week.
4. print(df) displays the weekly DataFrame stored in weekly.
5. resample("W") is an example of downsampling because the data goes from daily frequency to weekly frequency.

Answers

1. **True** — freq="D" means daily frequency.
2. **True** — after set_index("date"), the dates become the DataFrame index.
3. **True** — "W" specifies weekly frequency and .last() selects the last available value in each week.
4. **False** — print(df) prints the **original daily DataFrame**. To print the resampled data, use:
print(weekly)
5. **True** — you're reducing the number of observations from daily to weekly, so this is **downsampling**.

question 5:

import yfinance as yf

Download daily data
df = yf.download("AAPL",start="2025-01-01",end="2026-01-01")

Downsample daily data to weekly data
weekly = df.resample("W-FRI").agg({"Open": "first","High": "max","Low": "min","Close": "last"})
print(weekly)

True or False

1. yf.download("AAPL", ...) downloads historical market data for Apple.
2. resample("W-FRI") converts the daily data into weekly data, with weeks ending on Friday.
3. "High": "max" selects the highest price recorded during each week.
4. "Close": "first" selects the first closing price of each week.
5. Converting daily data into weekly data using resample() is an example of **downsampling**.

Answers

1. **True** — "AAPL" is Apple's ticker symbol.
2. **True** — W-FRI specifies a weekly frequency ending on Friday.
3. **True** — max selects the highest High value within each week.
4. **False** — the code uses:
   "Close": "last"
   so it selects the **last closing price** of each week.
5. **True** — you're reducing the frequency from daily observations to weekly observations.

Quizz - Time series

Question 6 :

```
import pandas as pd

weekly=pd.DataFrame(data=range(10,210,20),
        index=pd.date_range(start='2026-1-1',periods=10,freq='W'))
daily1=weekly.asfreq('D')
daily2=weekly.asfreq('D',method='bfill')
daily3=weekly.asfreq('D',method='ffill')
print(daily1)
print(daily2)
print(daily3)
```

True or False

1. range(10, 210, 20) creates 10 values starting at 10 and increasing by 20.
2. freq='W' creates a weekly date index.
3. weekly.asfreq('D') converts the weekly DataFrame to daily frequency and automatically fills the new values.
4. method='bfill' fills missing values using the **next available value**.
5. method='ffill' fills missing values using the **previous available value**.

Answers

1. **True** — the values are:
   10, 30, 50, 70, 90, 110, 130, 150, 170, 190
2. **True** — "W" means weekly frequency.
3. **False** — asfreq('D') creates a daily index, but without a filling method the newly created values are NaN.
4. **True** — bfill means **backward fill**, so it uses the next known value.
5. **True** — ffill means **forward fill**, so it uses the previous known value.

Question 7:

```
import pandas as pd

df = pd.DataFrame({

  "date": pd.date_range("2026-01-01", periods=10),

  "close": [100, 102, 101, 105, 107, 106, 110, 112, 111, 115]

})

df = df.set_index("date")

weekly = df.resample("W-FRI").agg({"open": "first","high": "max","low": "min","close": "last"})
```

True or False

1. df.set_index("date") makes the date column the index of the DataFrame.
2. "W-FRI" means that the data is resampled into weeks ending on Friday.
3. "close": "last" selects the last closing price available in each week.
4. The code will run successfully because pandas automatically creates the "open", "high", and "low" columns.

Quizz - Time series

5. If the DataFrame contained OHLC data, "high": "max" would select the highest value observed during each week.

Answers:

1. **True** — the dates become the DataFrame's index.
2. **True** — W-FRI means weekly frequency with Friday as the week-ending day.
3. **True** — .last() selects the last available close value in each weekly group.
4. **False** — open, high, and low don't exist in the DataFrame. This will cause a KeyError.
5. **True** — max finds the highest high value within each week.

Question 8 :

```
import yfinance as yf

stock = yf.download("AAPL", start="2025-01-01", end="2026-01-01")

print(df.head())

stock["MA20"] = stock["Close"].rolling(window=20).mean()
stock["rolling_std"] = stock["Close"].rolling(20).std()
```

True or False

1. yf.download("AAPL", start="2025-01-01", end="2026-01-01") downloads Apple's historical market data for the specified period.
2. stock["MA20"] = stock["Close"].rolling(window=20).mean() calculates a 20-observation moving average of the closing price.
3. rolling(20).std() calculates the standard deviation of the entire Close column, rather than a rolling standard deviation.
4. The first 19 values of MA20 will normally be NaN because there aren't yet 20 observations available.
5. print(df.head()) correctly displays the first five rows of the downloaded data.

Answers

1. **True** — yf.download() retrieves the historical data for the AAPL ticker between the specified dates.
2. **True** — rolling(window=20).mean() calculates the mean of each successive 20-observation window.
3. **False** — rolling(20).std() calculates the **rolling standard deviation**, using a new 20-observation window at each point.
4. **True** — by default, pandas requires 20 observations before calculating the first rolling mean.
5. **False** — the DataFrame is called stock, not df. It should be:

Question 9:

```
import pandas as pd

df = pd.DataFrame({
   "close": [100, 105, 102, 108, 110]
})
```

Quizz - Time series

```
df["close_lag1"] = df["close"].shift(1)
df["close_lag2"] = df["close"].shift(-1)
df["return"]=df.close-df.close_lag2
print(df)
```

True or False

1. df["close"].shift(1) creates a column containing the **previous observation's closing price**.
2. df["close"].shift(-1) creates a column containing the **next observation's closing price**.
3. The first value of close_lag1 will be NaN.
4. The last value of close_lag2 will be NaN.
5. df["return"] = df.close - df.close_lag2 calculates the percentage return of the stock.

Answers

1. **True** — shift(1) moves the values down by one row:

close	close_lag1
100	NaN
105	100
102	105
108	102
110	108

2. **True** — shift(-1) moves the values up by one row:

close	close_lag2
100	105
105	102
102	108
108	110
110	NaN

3. **True** — there is no previous observation for the first row.
4. **True** — there is no next observation for the last row.
5. **False** — it calculates a **difference in price** between the current price and the next price:

Question 10:

```
import pandas as pd

df = pd.DataFrame({
   "close": [100, 105, 102, 108, 110]
})

df["return"] = df["close"].pct_change()
df["cumulative_return"] = (1 + df["return"]).cumprod() – 1
```

True or False

1. df["close"].pct_change() calculates the percentage change between consecutive closing prices.
2. The first value of df["return"] will be NaN.
3. .cumprod() calculates the cumulative product of the values over time.
4. cumulative_return is calculated by simply adding all the individual returns together.

Quizz - Time series

5. If the initial price is 100 and the final price is 110, the cumulative return will be 10%.

Answers

1. **True** — pct_change() calculates the percentage change from one observation to the next.
2. **True** — there is no previous price for the first observation, so the first return is NaN.
3. **True** — cumprod() calculates the cumulative product.
4. **False** — returns are **compounded**, which is why we use:

(1 + df["return"]).cumprod() - 1

5. **True** — the price increased from 100 to 110, so the cumulative return is 10%.

Question 11:

```
import pandas as pd

df = pd.DataFrame({"AAPL": [100, 105, 110],"MSFT": [200, 210, 220]})

normalized = df.div(df.iloc[0])

performance = df.div(df.iloc[0]).mul(100)
```

True or False

1. df.iloc[0] selects the first row of the DataFrame.
2. df.div(df.iloc[0]) divides each column by its corresponding value in the first row.
3. The value of normalized["AAPL"].iloc[1] is 1.05.
4. performance contains values expressed as percentages, with the first row equal to 1 for both stocks.
5. normalized and performance contain the same values.

Answers

1. **True** — iloc[0] selects the first row:

AAPL 100

MSFT 200

2. **True** — pandas aligns the values by column:

   AAPL → divide by 100

   MSFT → divide by 200

3. **True** — for AAPL:

$$105/100 = 1.05$$

4. **False** — mul(100) converts the normalized values to percentage values, so the first row is **100**, not 1:

   AAPL MSFT

   0 100.0 100.0

5. **False** — performance is normalized × 100.

   For example:

Quizz - Time series

normalized:

	AAPL	MSFT
0	1.00	1.00
1	1.05	1.05
2	1.10	1.10

performance:

	AAPL	MSFT
0	100.0	100.0
1	105.0	105.0
2	110.0	110.0

Question 12:

```
Import pandas as pandas
df = pd.DataFrame({
   "AAPL": [100, 105, 110],
   "MSFT": [200, 210, 220]
})

shares = pd.Series({
   "AAPL": 10,
   "MSFT": 5
})
position_value = df.mul(shares)
print(position_value)
```

True or False

1. shares is a pandas Series containing the number of shares held for each stock.
2. df.mul(shares) multiplies each stock price by the corresponding number of shares.
3. The value of position_value["AAPL"].iloc[0] is 1,000.
4. The value of position_value["MSFT"].iloc[2] is 1,100.
5. Pandas matches the values in shares to the columns of df using their labels (AAPL and MSFT).

Answers

1. **True** — shares contains:
   AAPL 10
   MSFT 5
2. **True** — pandas aligns the Series with the DataFrame columns:
   AAPL → price × 10
   MSFT → price × 5
3. **True** —
$$100 \times 10 = 1000$$
4. **True** —

$$220 \times 5 = 1100$$

5. **True** — this is one of the useful features of pandas. It uses the labels to perform the multiplication.